ZIM School

Entity Relationship Diagram and Data Model Guide

43 entities (tables) · 17 enumerations · 434 columns · 13 modules
Generated from the Doctrine entity mappings in `src/*/Domain/Model` on 2026-11-10.

1. Purpose of this document

This guide explains what ZIM School stores and why. It shows every database table (each one is a Doctrine entity in a module's `Domain/Model` folder), how the tables connect, and how data flows through a school year, from setting up a school to printing report cards.

Read it in three passes: (1) the big picture in sections 2-4, (2) the module chapters in section 5, each with its own diagram and a column-by-column dictionary, and (3) the reference lists at the end (enumerations, delete behaviour, glossary).

How to read the diagrams

Lines use the standard crow's-foot symbols, and each end also carries a label saying how many records take part on that side.

Line end	Label	Meaning
Two bars	**1**	Exactly one: the foreign key is required.
Bar + circle	**0..1**	Zero or one: the foreign key is optional, or the side of a one-to-one link that may be missing.
Crow's foot + circle	*****	Zero or many.

Read a line from one end to the other: **School 1 —— * Section** means one school has many sections and every section belongs to exactly one school. **User 0..1 —— * Section** (adviser) means a section may have no adviser. **User 1 —— 0..1 Learner** means each learner has exactly one account and an account belongs to at most one learner.

Other symbol	Meaning
PK / FK / UK	Primary key / foreign key / unique column.
Label on a line	The property name on the child entity (the column is that name in snake_case + `_id`).
Dashed box with only `id`	A table owned by another module (its name is under the title), shown so the links are visible.
+ N more columns	The table has N more columns than the diagram shows; see the dictionary table.

Colours

Every module has its own colour. A table's title bar has its module's colour, and a relationship line has the colour of the module of the table it points to (its 1 or 0..1 side), so you can see at a glance which module each link reaches into. In the module map the arrow has the colour of the module that does the using. Each diagram lists the colours it uses.

Core · Access · Billing · Schools · Classes · Learners · Attendance · Grading · Nutrition · LessonPlans · Staff · Announcements · Dashboards

> All primary keys are auto-increment integers named `id`. Table names are the class names in snake_case (e.g. `ScheduleEntry` → `schedule_entry`). Enumerations are stored as short strings (or small integers for `GradeLevel` and `Weekday`).

2. The system at a glance

ZIM School is a school grading platform for Philippine DepEd schools, built with Symfony 8.1 / PHP 8.4, Doctrine ORM on MySQL, Twig, Stimulus/Turbo and some Vue. It covers the paperwork of a school year: enrollment and the School Register (SF1), daily attendance (SF2), nutritional status (SF8), grades and report cards (card slips), weekly lesson plans, learner and staff ID cards, the Personal Data Sheet (PDS) and the service record.

Who uses it

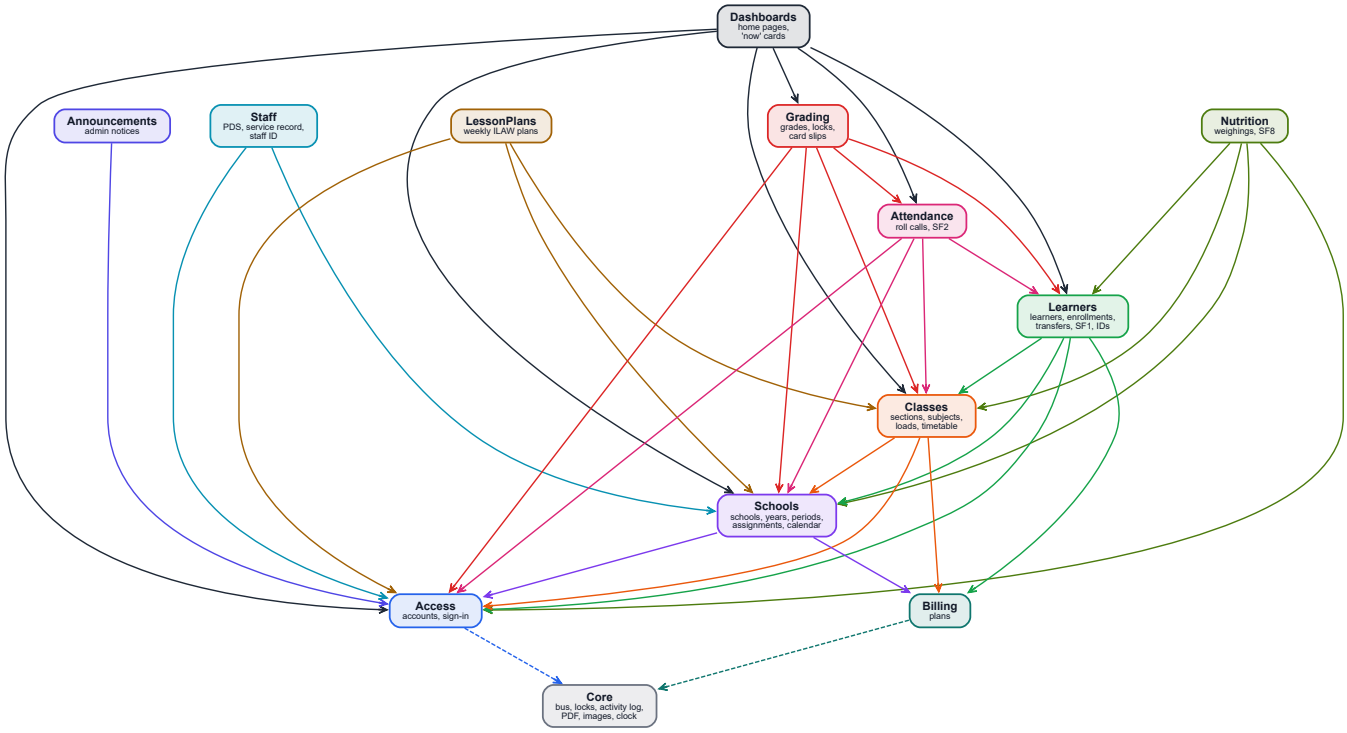
Role	What they do
Administrator	Runs the platform: plans, divisions, schools, school years, accounts, principal assignments, announcements; can act on any school.
Principal	Runs one school per school year: profile, teachers, calendar, class periods, sections, subjects, timetable, enrollment, transfers, grading openings and locks, change requests, staff records.
Teacher	Teaches loads (grade sheets, lesson plans). As a section adviser: class list, attendance and SF2, weighings and SF8, card slips. On the Individual plan (no principal) the teacher also sets up their own section and subjects.
Student	Signs in with the LRN to see their enrollment and their section's timetable.

How the code is organised

The application is a **modular monolith** with Domain-Driven Design layers. Each module in `src/` has `Domain` (entities, repository interfaces, rules), `Application` (use cases, queries, services, public API), `Infrastructure` (Doctrine repositories, PDF/spreadsheet writers) and `Presentation` (controllers, forms, Twig templates, Stimulus/Vue). Every change goes through the **use case bus** (one command, one handler, one database transaction, with row-level *transaction locks* so two people can't overwrite each other); every page read goes through the **query bus** (optionally cached). Modules may reference each other's entities, but call each other only through public API interfaces. Deptrac enforces these rules.

3. Modules and how they depend on each other

Arrows point from a module to the modules whose public API it uses (taken from the deptrac rules in `config/deptrac.yaml`; every module also uses Core and Access's `User`). Lower modules never call upper ones directly; when a lower module needs data owned by an upper one (e.g. Schools must ask Grading whether a period has grades) it defines a *hook interface* that the upper module implements.



Core · Access · Billing · Schools · Classes · Learners · Attendance · Grading · Nutrition · LessonPlans · Staff · Announcements · Dashboards

Arrow colour = the module that uses the other one; dashed = Core, used by every module.

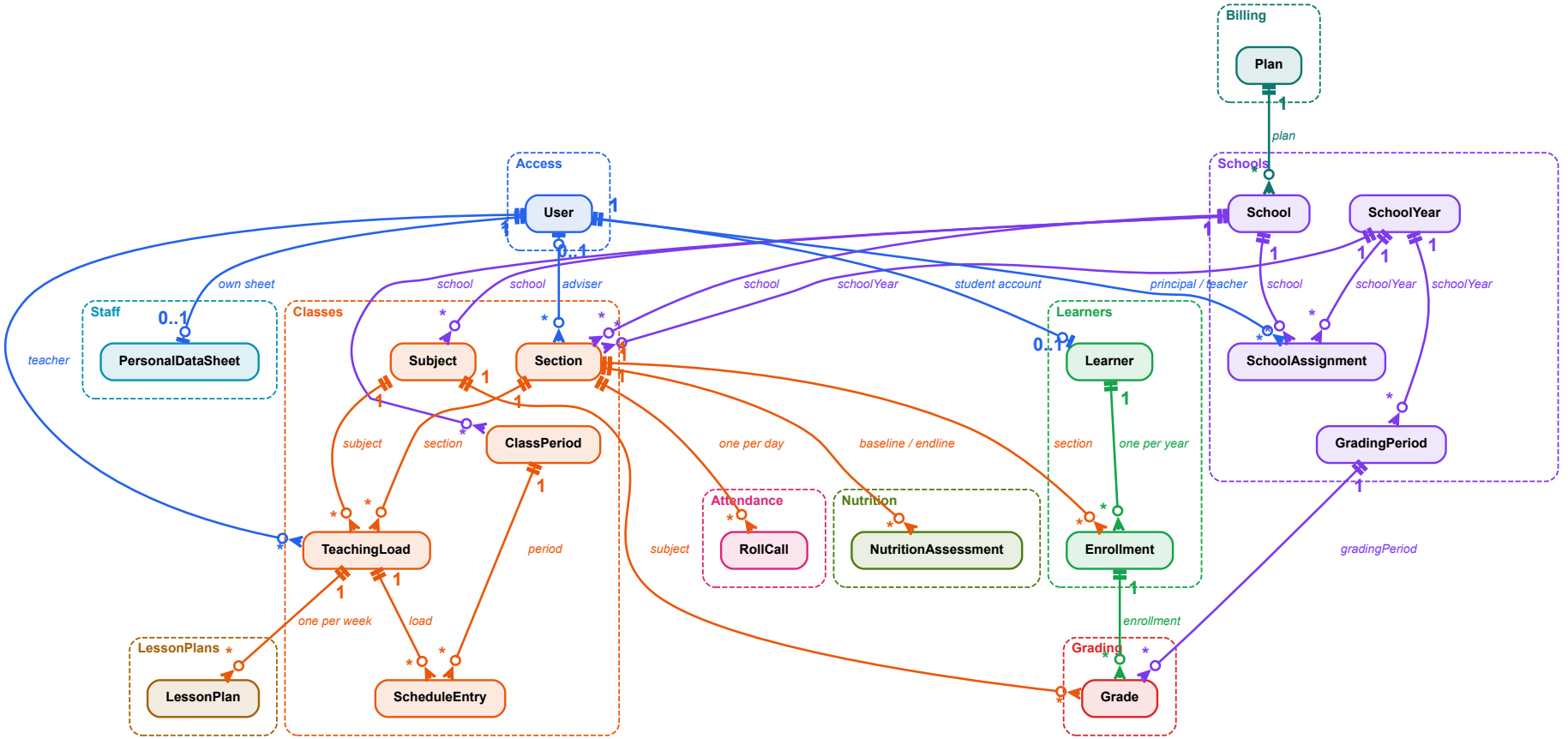
Module	Entities	Responsibility
Core	ActivityLog	Bus, transaction locks, activity feed, school clock, image and PDF services, layouts.
Access	User	Accounts, sign-in, password changes, profile photos, admin account screens.
Billing	Plan	Subscription plans and their teacher/student limits per seat.
Schools	School, SchoolYear, GradingPeriod, SchoolAssignment, SchoolDayOff, Division	Schools, years, principals/teachers per year, calendar, plan limits.
Classes	Section, Subject, ClassPeriod, TeachingLoad, ScheduleEntry	Sections, subjects, bell schedule, teaching loads and timetable.
Learners	Learner, Enrollment, TransferRequest	Learners, enrollments, transfers, SF1, learner ID cards.
Attendance	RollCall, AttendanceMark, Sf2Submission	Daily roll calls and the monthly SF2.
Grading	Grade, GradeOpening, GradeLock, GradeUnlock, GradeChangeRequest, GradeChangeLine, CardSlipComment	Grades, grading control, change requests and card slips.

■ **Nutrition**	NutritionAssessment, NutritionMeasurement, Sf8Submission	Baseline/endline weighings and the SF8.
■ **LessonPlans**	LessonPlan	Weekly ILAW lesson plans and their PDF.
■ **Staff**	PersonalDataSheet + 7 PDS lists, ServiceRecord, ServiceRecordEntry, StaffIdDetails	PDS, service record and staff ID.
■ **Announcements**	Announcement	Admin announcements.
■ **Dashboards**	(none)	The four role dashboards, built from the other modules' public APIs.

4. The big picture

4.1 Core relationships (simplified)

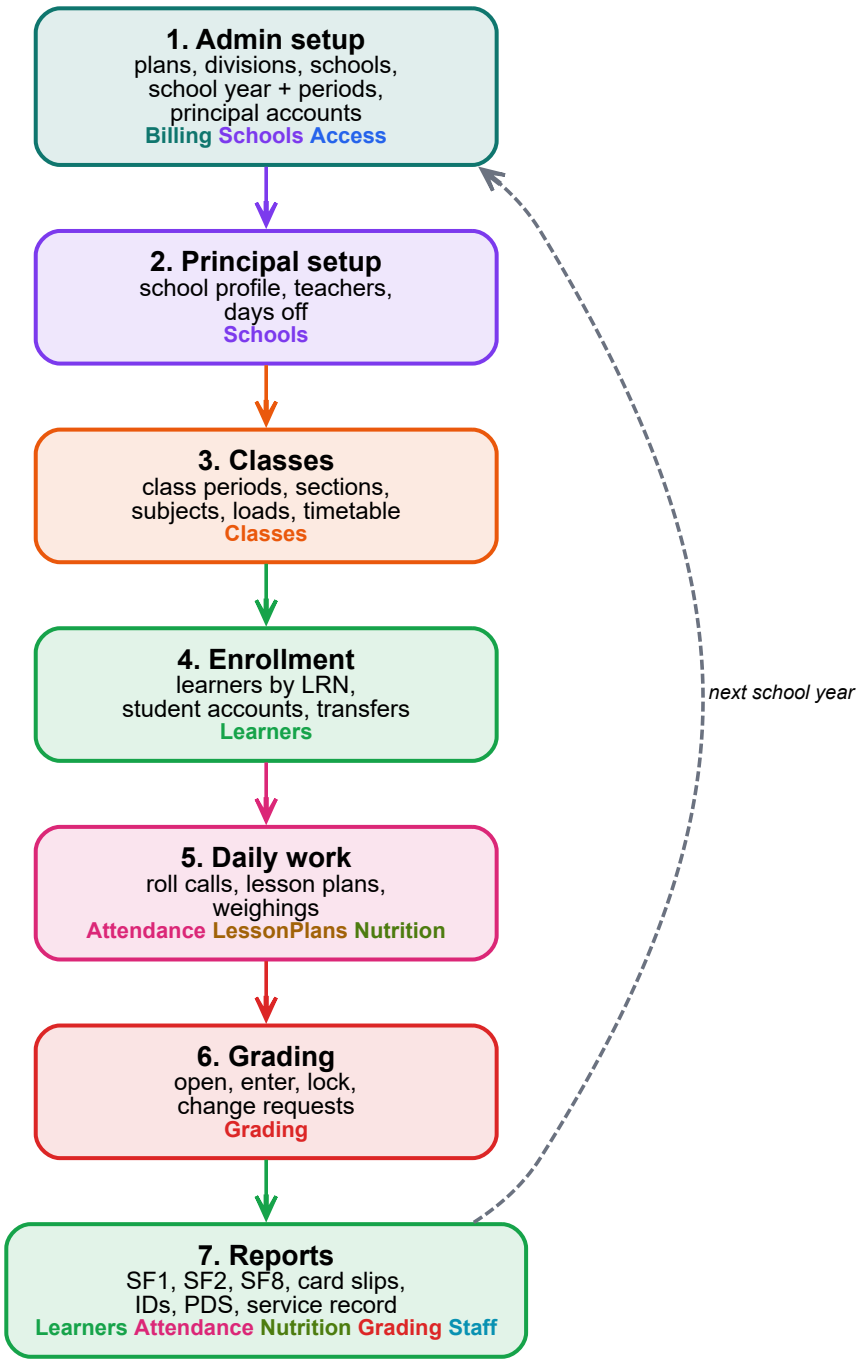
The backbone of the model, without columns and without audit links ("who did it" users). Almost every record hangs from a **School** and a **SchoolYear**; learner data hangs from the **Enrollment**; teaching data hangs from the **TeachingLoad**. Dashed frames group the tables by module.



Billing · Schools · Access · Classes · Learners · Grading · Attendance · Nutrition · LessonPlans · Staff — Line colour = the module of the table the line points to (its 1 side).

4.2 The flow of a school year

The order in which data is created. Each step needs the records of the step before it.



Billing · Schools · Access · Classes · Learners · Attendance · LessonPlans · Nutrition · Grading · Staff — Box colour = the main module of the step; the module names under each step say which modules hold its records.

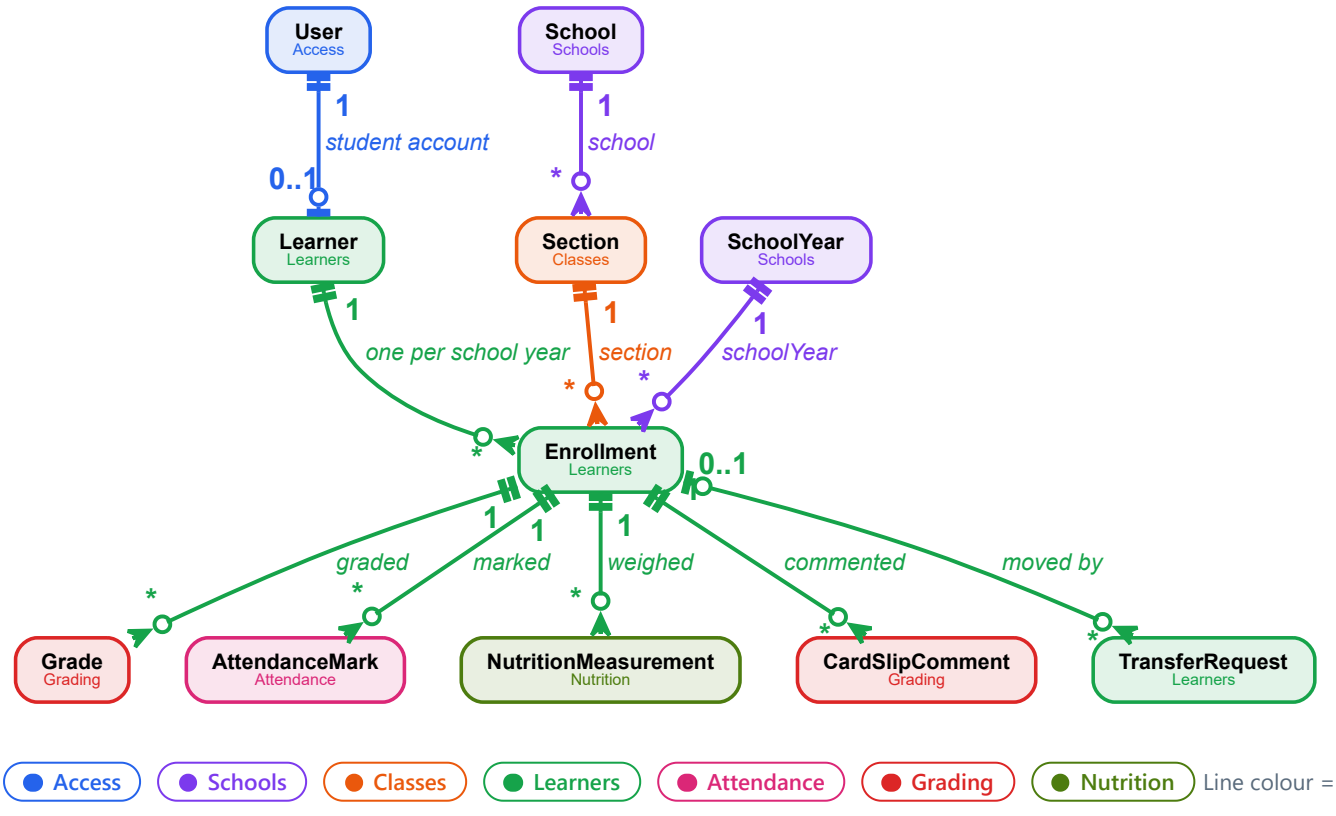
Step	Who	Records created
1. Admin setup	Administrator	Plan, Division, School, SchoolYear (+ GradingPeriods), User (principal), SchoolAssignment (principal)

2. Principal setup	Principal	School profile fields, User + SchoolAssignment (teachers), SchoolDayOff, ClassPeriod
3. Classes	Principal (or the teacher on the Individual plan)	Section (with adviser), Subject, TeachingLoad, ScheduleEntry
4. Enrollment	Principal or adviser	Learner + User (student), Enrollment, TransferRequest
5. Daily work	Adviser / teacher	RollCall + AttendanceMark, LessonPlan, NutritionAssessment + NutritionMeasurement
6. Grading	Principal and teachers	GradeOpening, Grade, GradeLock, GradeUnlock, GradeChangeRequest + lines, CardSlipComment
7. Reports and submissions	Adviser, principal	Sf2Submission, Sf8Submission; files (SF1/SF2/SF8 spreadsheets, PDFs) are generated on demand and not stored

> At the start of a new school year the admin creates the year, makes it current and copies the principals; principals bring back last year's teachers, sections and subjects with one click each. Learners are permanent, so returning learners are simply enrolled again by LRN.

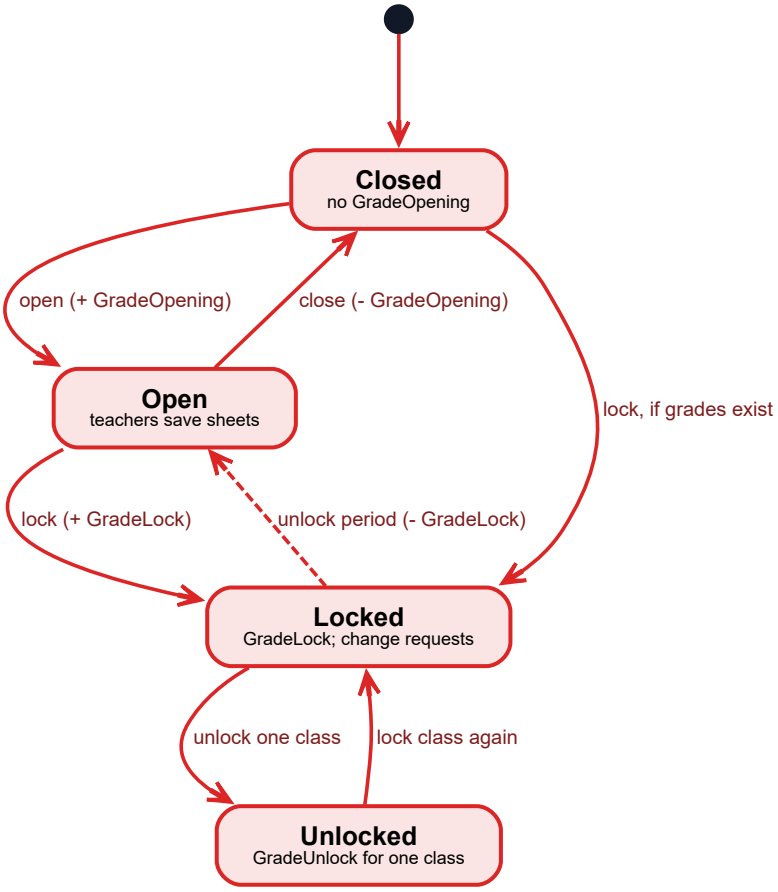
4.3 Everything about a learner hangs from the enrollment

Grades, attendance marks, measurements and card slip comments point to the **Enrollment** (learner + year), not to the learner. That keeps each year's records separate and lets a transfer simply move the enrollment to another section.



4.4 The life of a grading period at one school

The period's state is not a column: it is derived from which control rows exist (`ClassGradeStatus` = Open, Closed, Locked or Unlocked). The principal or an admin opens and closes; only the principal locks and unlocks. After the period is unlocked it is Open or Closed again, depending on whether its GradeOpening still exists.

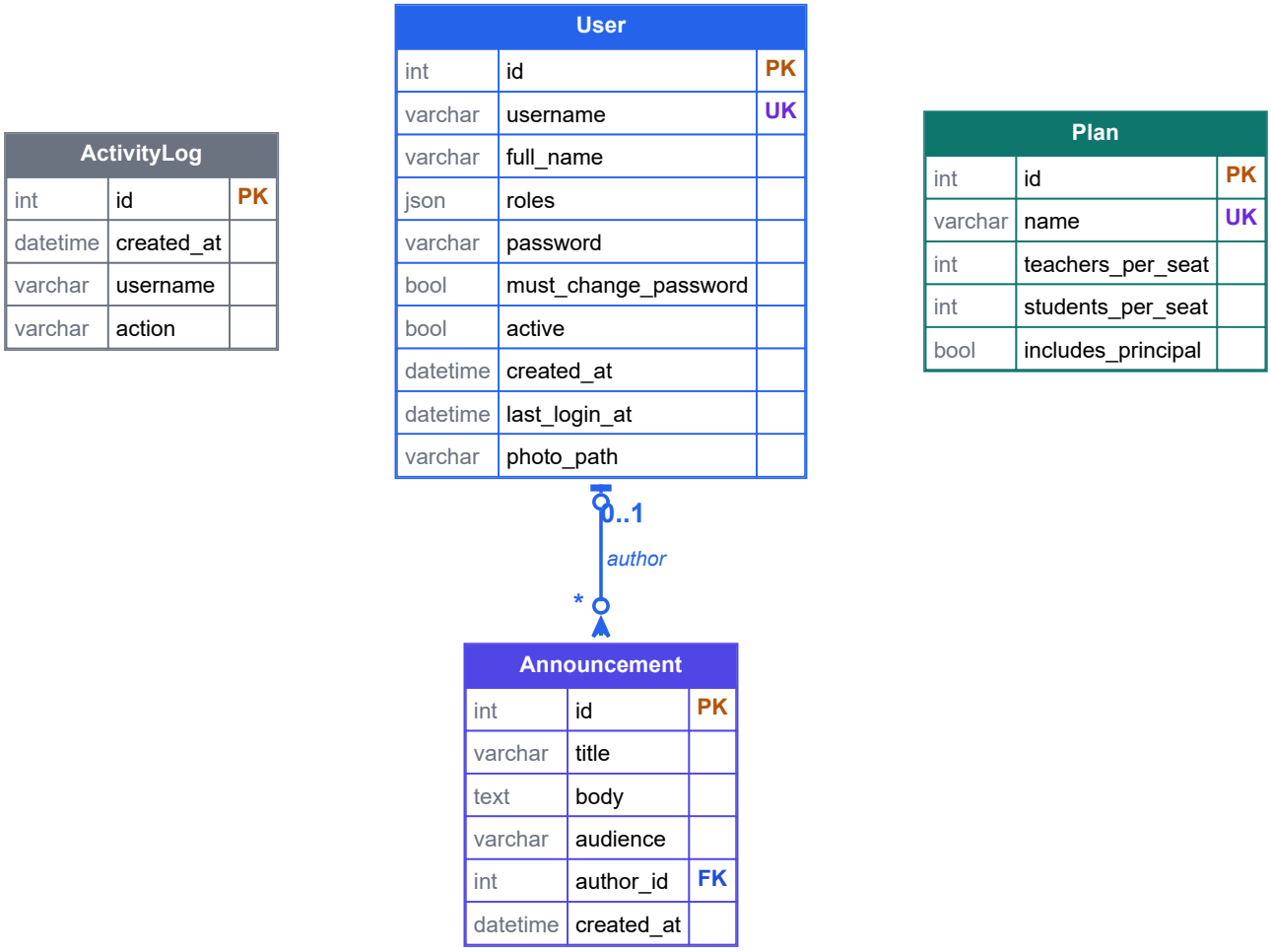


Grading All grading control rows belong to the Grading module.

5. Module chapters

5.1 Platform: accounts, plans, activity and announcements

The foundation every other module stands on. **User** is the single account table for all four roles. **Plan** (Billing) sets how many teachers and students a school may have. **ActivityLog** (Core) is the audit feed shown on the admin dashboard, and **Announcement** holds notices the admin posts.



Core · Access · Billing · Announcements — Line colour = the module of the table the line points to (its 1 side).

User table `user` · Access

Every sign-in account: administrator, principal, teacher or student. Holds exactly one role, the hashed password, whether the password must be changed at the next sign-in, and an optional profile photo.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`username`	VARCHAR(50)	UK	no	
`full_name`	VARCHAR(120)		no	
`roles`	JSON		no	
`password`	VARCHAR(255)		no	
`must_change_password`	BOOLEAN		no	
`active`	BOOLEAN		no	
`created_at`	DATETIME		no	
`last_login_at`	DATETIME		yes	
`photo_path`	VARCHAR(255)		yes	

Announcement table `announcement` · Announcements

A notice the administrator posts for an audience (everyone, principals, teachers or students). The author is kept as a link that becomes empty if the account is deleted.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`title`	VARCHAR(150)		no	
`body`	TEXT		no	
`audience`	VARCHAR(20)		no	
`author_id`	INT	FK	yes	→ User (on delete SET NULL)
`created_at`	DATETIME		no	

Plan table `plan` · Billing

What a school bought: how many teachers and students each seat allows, and whether the plan includes a principal (Basic does, Individual does not).

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`name`	VARCHAR(50)	UK	no	
`teachers_per_seat`	INT		no	
`students_per_seat`	INT		no	
`includes_principal`	BOOLEAN		no	

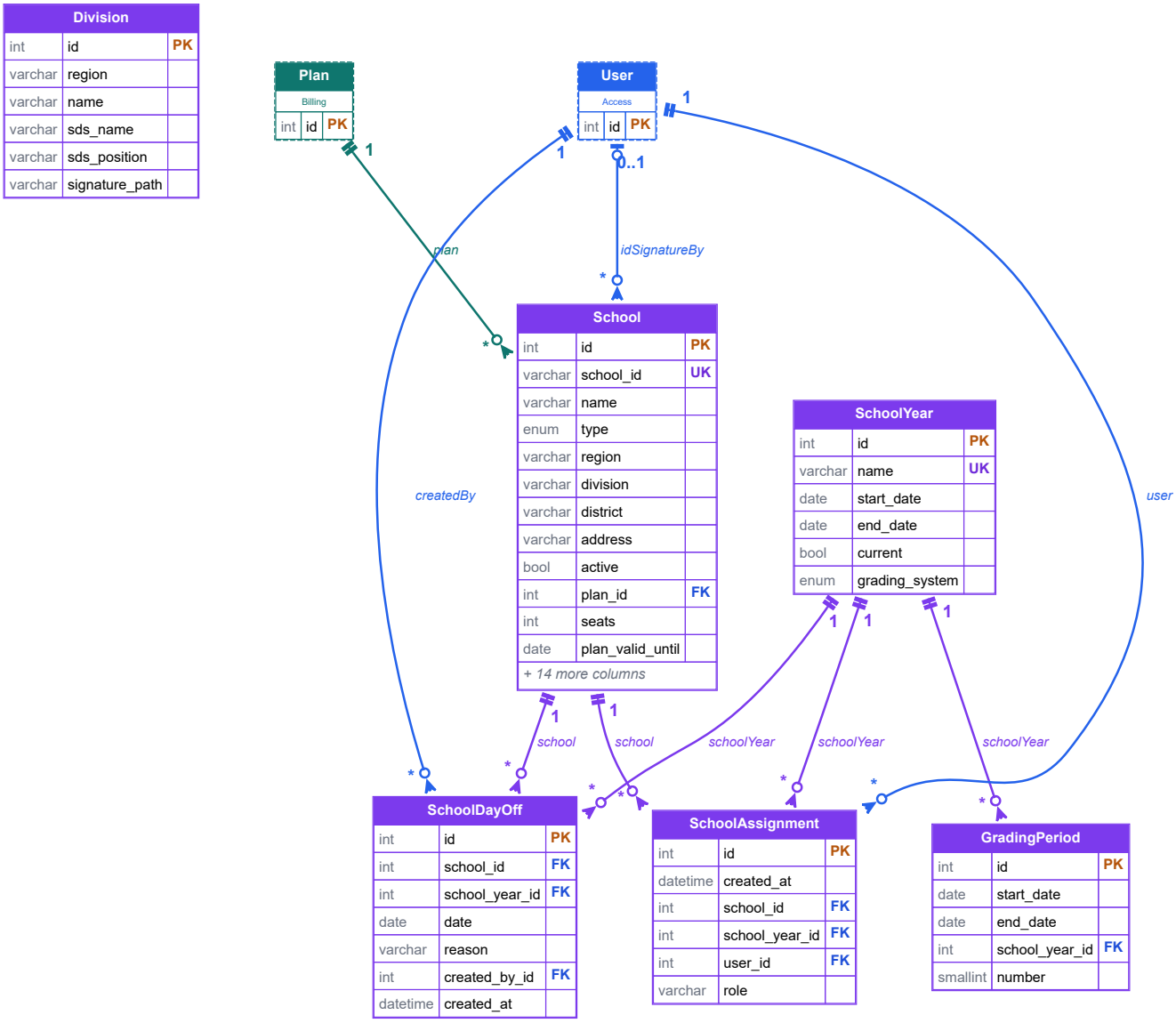
■ ActivityLog table `activity_log` · Core

One line of the admin's "Recent activity" feed. The username is copied as text (not a foreign key) so history survives account changes.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`created_at`	DATETIME		no	
`username`	VARCHAR(50)		no	
`action`	VARCHAR(255)		no	

5.2 Schools: schools, school years, assignments and the calendar

Defines *where* and *when*. A **School** runs on a **Plan**; a **SchoolYear** (shared by every school, one is current) owns its **GradingPeriods**. **SchoolAssignment** says who is the principal or a teacher of a school in a given year; **SchoolDayOff** marks holidays. **Division** holds the Superintendent who signs principals' IDs.



Access · Billing · Schools — Line colour = the module of the table the line points to (its 1 side).

Division table `division` · Schools

A schools division and its Superintendent (SDS), whose name and signature print on principals' IDs. Matched to schools by region + division name.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`region`	VARCHAR(50)		no	
`name`	VARCHAR(100)		no	
`sds_name`	VARCHAR(150)		yes	
`sds_position`	VARCHAR(100)		no	
`signature_path`	VARCHAR(255)		yes	

Unique rules: `region, name`

GradingPeriod table `grading_period` · Schools

A term or quarter of a school year (number 1..n) with its dates. Each school opens it for grading on its own.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`start_date`	DATE		yes	
`end_date`	DATE		yes	
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`number`	SMALLINT		no	

Unique rules: `school_year_id, number`

School table `school` · Schools

A DepEd school (6-digit School ID). Carries its plan, number of seats and plan expiry, its region/division/district as text, logo and ID signature, and the names that print on SF8, service records and lesson plans.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`school_id`	VARCHAR(6)	UK	no	
`name`	VARCHAR(150)		no	
`type`	VARCHAR(20) enum SchoolType		no	
`region`	VARCHAR(100)		no	
`division`	VARCHAR(100)		no	
`district`	VARCHAR(100)		yes	
`address`	VARCHAR(255)		yes	
`active`	BOOLEAN		no	
`plan_id`	INT	FK	no	→ Plan
`seats`	INT		no	default 1
`plan_valid_until`	DATE		yes	
`logo_path`	VARCHAR(255)		yes	
`id_signature_path`	VARCHAR(255)		yes	
`id_signature_by_id`	INT	FK	yes	→ User (on delete SET NULL)
`contact_number`	VARCHAR(30)		yes	
`email`	VARCHAR(180)		yes	
`facebook_url`	VARCHAR(255)		yes	
`sf8_certifier_name`	VARCHAR(100)		yes	
`sf8_certifier_position`	VARCHAR(100)		yes	
`sr_prepared_by_name`	VARCHAR(100)		yes	
`sr_prepared_by_designation`	VARCHAR(100)		yes	
`sr_certified_by_name`	VARCHAR(100)		yes	
`sr_certified_by_designation`	VARCHAR(100)		yes	
`lp_checked_by_name`	VARCHAR(100)		yes	
`lp_checked_by_position`	VARCHAR(100)		yes	

SchoolAssignment table `school_assignment` · Schools

Who works at which school in which school year, and as what (principal or teacher). One assignment per user per school year.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`created_at`	DATETIME		no	
`school_id`	INT	FK	no	→ School
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`user_id`	INT	FK	no	→ User
`role`	VARCHAR(30)		no	

Unique rules: `user_id, school_year_id`

SchoolDayOff table `school_day_off` · Schools

A weekday without classes at a school (holiday, suspension). Used by attendance, lesson plans and the calendar.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`school_id`	INT	FK	no	→ School (on delete CASCADE)
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`date`	DATE		no	
`reason`	VARCHAR(100)		no	
`created_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`created_at`	DATETIME		no	

Unique rules: `school_id, date`

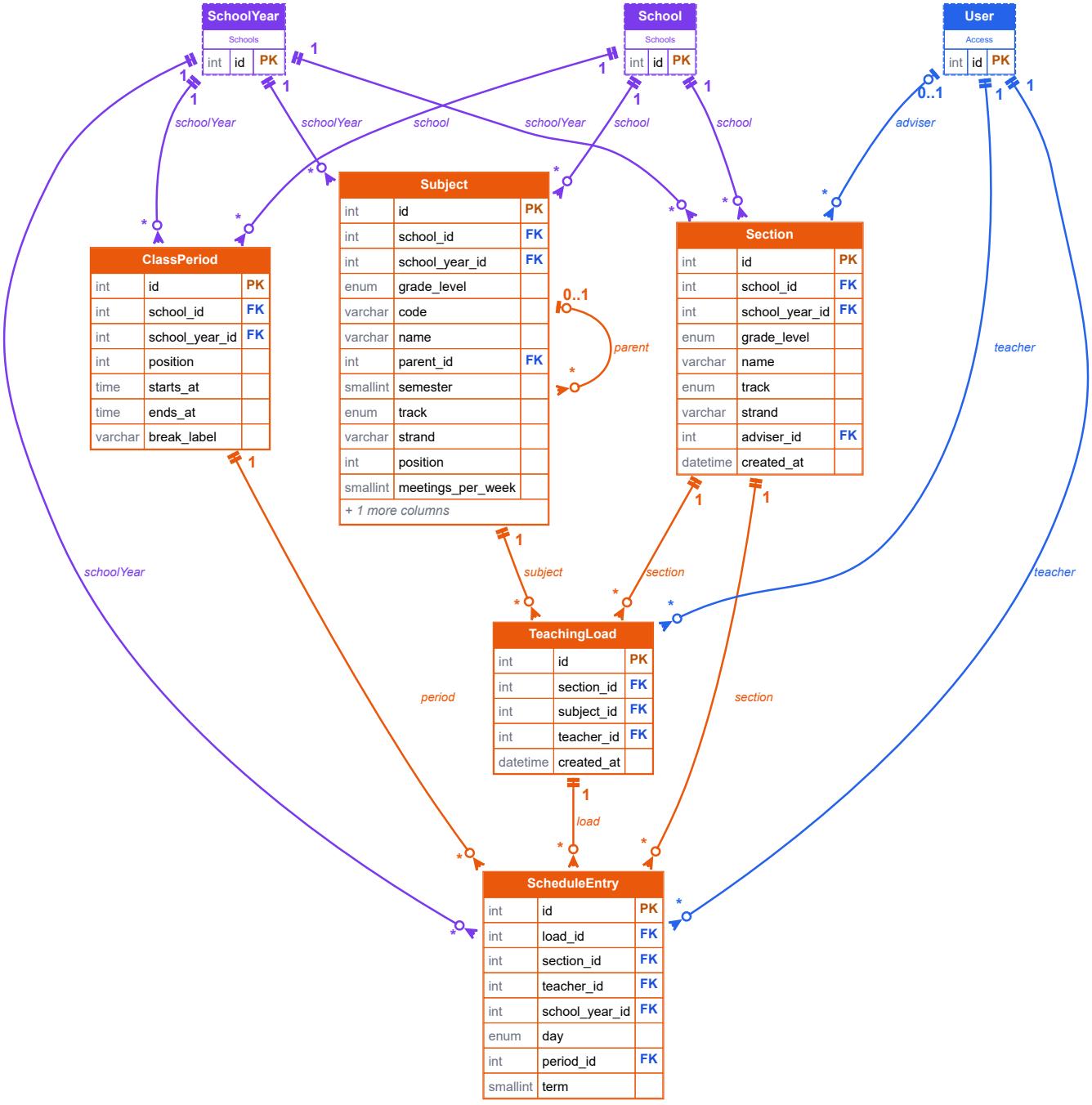
SchoolYear table `school_year` · Schools

A school year shared by all schools (e.g. 2026-2027). Exactly one is current. Uses either terms or quarters (GradingSystem) and owns its grading periods.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`name`	VARCHAR(20)	UK	no	
`start_date`	DATE		no	
`end_date`	DATE		no	
`current`	BOOLEAN		no	
`grading_system`	VARCHAR(10) enum GradingSystem		no	

5.3 Classes: sections, subjects, teaching loads and the timetable

Defines *what is taught, to whom, by whom and when*. Each school, per school year, has **Sections**, **Subjects** (with optional components such as MAPEH), and **ClassPeriods** (the bell schedule). A **TeachingLoad** pairs a section with a subject and a teacher; its **ScheduleEntries** place it on the weekly timetable.



Access · Schools · Classes — Line colour = the module of the table the line points to (its 1 side).

ClassPeriod table `class_period` · Classes

A time slot of the school day (07:30-08:30) for a school and year. A slot with a break label (Recess, Lunch) is not a class period.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`school_id`	INT	FK	no	→ School (on delete CASCADE)
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`position`	INT		no	
`starts_at`	TIME		no	
`ends_at`	TIME		no	
`break_label`	VARCHAR(40)		yes	

ScheduleEntry table `schedule_entry` · Classes

One meeting of a teaching load on the timetable: weekday + class period (+ term for semester subjects). Unique per section slot and per teacher slot, so no double booking.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`load_id`	INT	FK	no	→ TeachingLoad (on delete RESTRICT)
`section_id`	INT	FK	no	→ Section (on delete RESTRICT)
`teacher_id`	INT	FK	no	→ User (on delete RESTRICT)
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`day`	SMALLINT enum Weekday		no	
`period_id`	INT	FK	no	→ ClassPeriod (on delete RESTRICT)
`term`	SMALLINT		no	

Unique rules: `section_id, day, period_id, term`; `teacher_id, school_year_id, day, period_id, term`

Section table `section` · Classes

A class (e.g. Grade 8 - ARTISTIC) of a school in one school year, with an optional adviser (a teacher may advise only one section per year). SHS sections carry a track and strand.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`school_id`	INT	FK	no	→ School (on delete CASCADE)
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`grade_level`	SMALLINT enum GradeLevel		no	
`name`	VARCHAR(60)		no	
`track`	VARCHAR(20) enum ShsTrack		yes	
`strand`	VARCHAR(60)		yes	
`adviser_id`	INT	FK	yes	→ User (on delete SET NULL)
`created_at`	DATETIME		no	

Unique rules: `school_id, school_year_id, grade_level, name`; `school_year_id, adviser_id`

Subject table `subject` · Classes

A subject a school offers to one grade level in a school year. A subject with a parent is a component (e.g. Music, Arts of MAPEH); components carry the grades, the parent's grade is computed.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`school_id`	INT	FK	no	→ School (on delete CASCADE)
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`grade_level`	SMALLINT enum GradeLevel		no	
`code`	VARCHAR(20)		no	
`name`	VARCHAR(100)		no	
`parent_id`	INT	FK	yes	→ Subject (on delete CASCADE)
`semester`	SMALLINT		yes	
`track`	VARCHAR(20) enum ShsTrack		yes	
`strand`	VARCHAR(60)		yes	
`position`	INT		no	
`meetings_per_week`	SMALLINT		yes	
`created_at`	DATETIME		no	

Unique rules: `school_id, school_year_id, code`

TeachingLoad table `teaching_load` · Classes

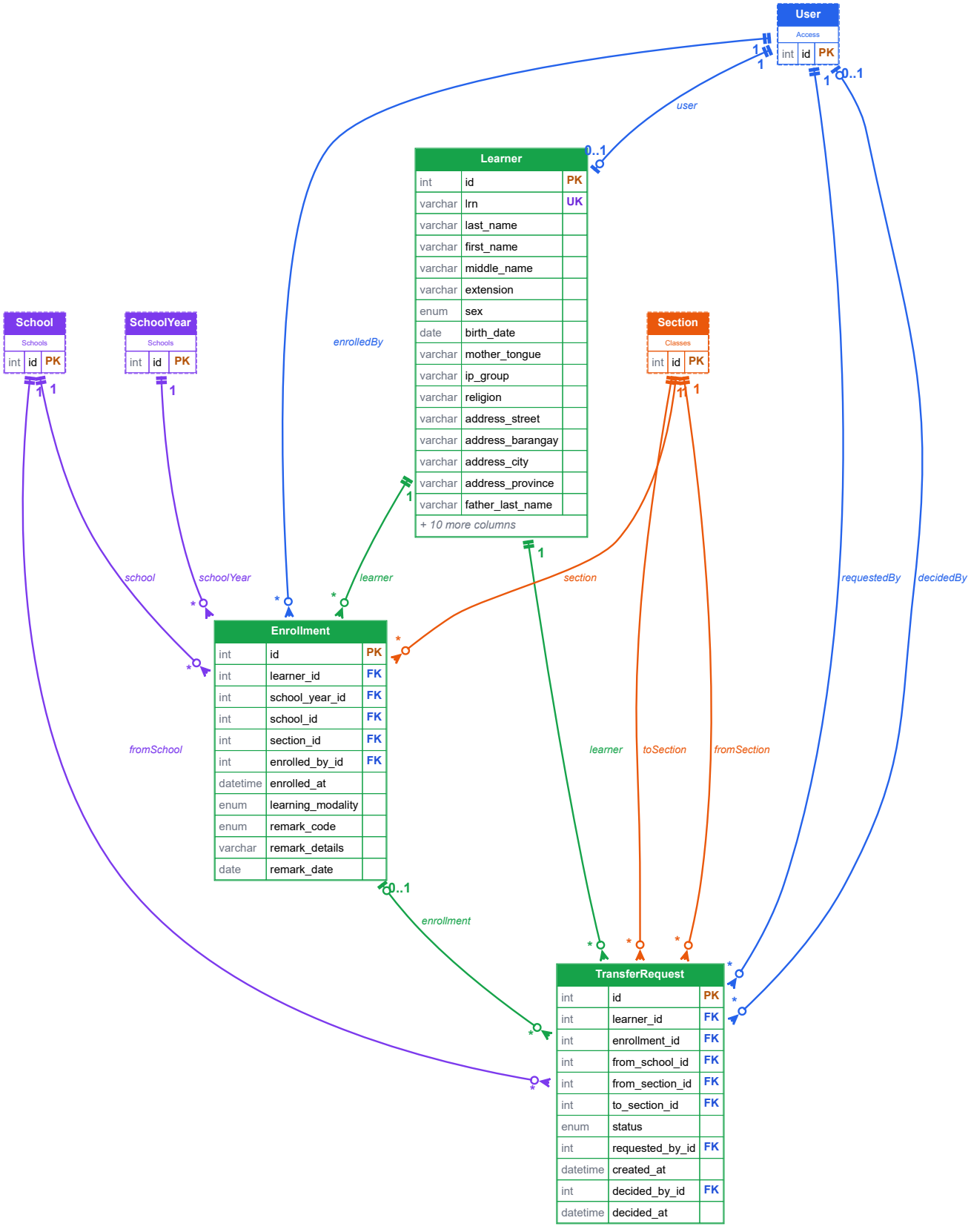
A teacher assigned to teach one subject in one section. Grades, the timetable and lesson plans all hang from it.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`section_id`	INT	FK	no	→ Section (on delete RESTRICT)
`subject_id`	INT	FK	no	→ Subject (on delete RESTRICT)
`teacher_id`	INT	FK	no	→ User (on delete RESTRICT)
`created_at`	DATETIME		no	

Unique rules: `section_id, subject_id`

5.4 Learners: learners, enrollments and transfers

A **Learner** is permanent (identified by the LRN) and owns a student account. Each school year the learner gets one **Enrollment** in a section. A **TransferRequest** moves an enrolled learner to another school once the school being left approves.



Access · Schools · Classes · Learners — Line colour = the module of the table the line points to (its 1 side).

Enrollment table `enrollment` · Learners

A learner's place in one section for one school year (one per learner per year), with the SF1 learning modality and remarks.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`learner_id`	INT	FK	no	→ Learner (on delete RESTRICT)
`school_year_id`	INT	FK	no	→ SchoolYear (on delete CASCADE)
`school_id`	INT	FK	no	→ School (on delete RESTRICT)
`section_id`	INT	FK	no	→ Section (on delete RESTRICT)
`enrolled_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`enrolled_at`	DATETIME		no	
`learning_modality`	VARCHAR(20) enum LearningModality		no	default 'face_to_face'
`remark_code`	VARCHAR(20) enum Sf1Remark		yes	
`remark_details`	VARCHAR(150)		yes	
`remark_date`	DATE		yes	

Unique rules: `learner_id, school_year_id`

Learner table `learner` · Learners

A child, kept for life across schools and years, identified by the 12-digit LRN. Owns a student login account (username = LRN).

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`lrn`	VARCHAR(12)	UK	no	
`last_name`	VARCHAR(50)		no	
`first_name`	VARCHAR(50)		no	
`middle_name`	VARCHAR(50)		yes	
`extension`	VARCHAR(10)		yes	
`sex`	VARCHAR(10) enum SexAtBirth		no	
`birth_date`	DATE		no	
`mother_tongue`	VARCHAR(50)		yes	
`ip_group`	VARCHAR(50)		yes	
`religion`	VARCHAR(50)		yes	
`address_street`	VARCHAR(100)		yes	
`address_barangay`	VARCHAR(50)		yes	
`address_city`	VARCHAR(50)		yes	
`address_province`	VARCHAR(50)		yes	
`father_last_name`	VARCHAR(50)		yes	
`father_first_name`	VARCHAR(50)		yes	
`father_middle_name`	VARCHAR(50)		yes	
`mother_last_name`	VARCHAR(50)		yes	
`mother_first_name`	VARCHAR(50)		yes	
`mother_middle_name`	VARCHAR(50)		yes	
`guardian_name`	VARCHAR(100)		yes	
`guardian_relationship`	VARCHAR(30)		yes	
`contact_number`	VARCHAR(20)		yes	
`user_id`	INT	FK	no	→ User (on delete RESTRICT)
`created_at`	DATETIME		no	

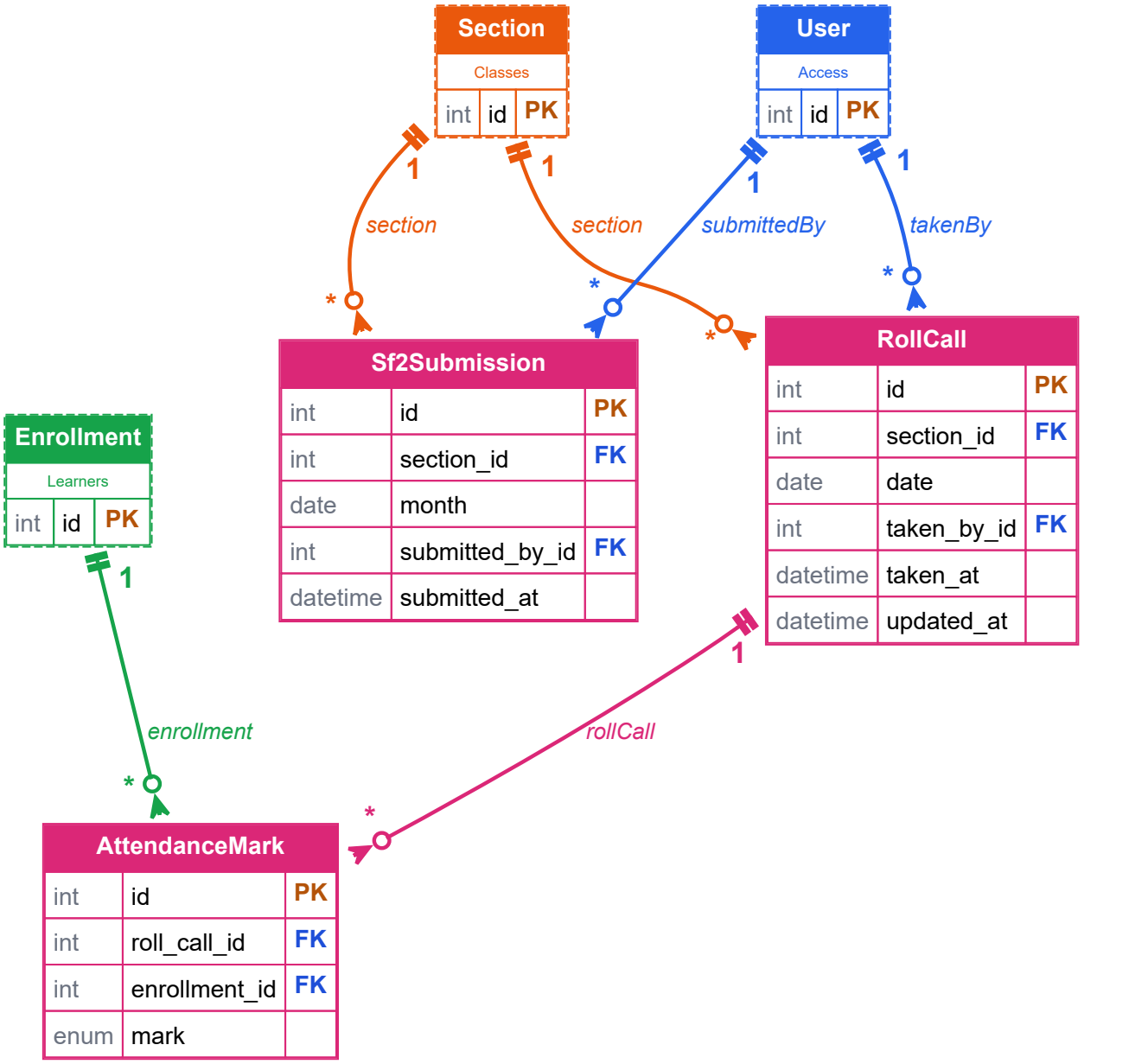
TransferRequest table `transfer_request` · Learners

A receiving school's request to move an enrolled learner into one of its sections. The school being left approves or declines; the receiver may cancel.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`learner_id`	INT	FK	no	→ Learner (on delete RESTRICT)
`enrollment_id`	INT	FK	yes	→ Enrollment (on delete SET NULL)
`from_school_id`	INT	FK	no	→ School (on delete RESTRICT)
`from_section_id`	INT	FK	no	→ Section (on delete CASCADE)
`to_section_id`	INT	FK	no	→ Section (on delete CASCADE)
`status`	VARCHAR(10) enum TransferStatus		no	
`requested_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`created_at`	DATETIME		no	
`decided_by_id`	INT	FK	yes	→ User (on delete SET NULL)
`decided_at`	DATETIME		yes	

5.5 Attendance: roll calls and SF2

The adviser takes a **RollCall** per school day; only exceptions are stored as **AttendanceMarks** (absent, late, cutting). At month end the adviser submits the SF2; the **Sf2Submission** locks that month until the principal reopens it.



Access · Classes · Learners · Attendance — Line colour = the module of the table the line points to (its 1 side).

AttendanceMark table `attendance_mark` · Attendance

One learner absent, late or cutting classes in a roll call.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`roll_call_id`	INT	FK	no	→ RollCall (on delete CASCADE)
`enrollment_id`	INT	FK	no	→ Enrollment (on delete CASCADE)
`mark`	VARCHAR(10) enum LearnerMark		no	

Unique rules: `roll_call_id, enrollment_id`

RollCall table `roll_call` · Attendance

A section's attendance for one school day. Only learners who were not simply present get a mark.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`section_id`	INT	FK	no	→ Section (on delete CASCADE)
`date`	DATE		no	
`taken_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`taken_at`	DATETIME		no	
`updated_at`	DATETIME		yes	

Unique rules: `section_id, date`

Sf2Submission table `sf2_submission` · Attendance

The adviser's SF2 (Daily Attendance) for one month handed to the principal. While it exists, that month is locked.

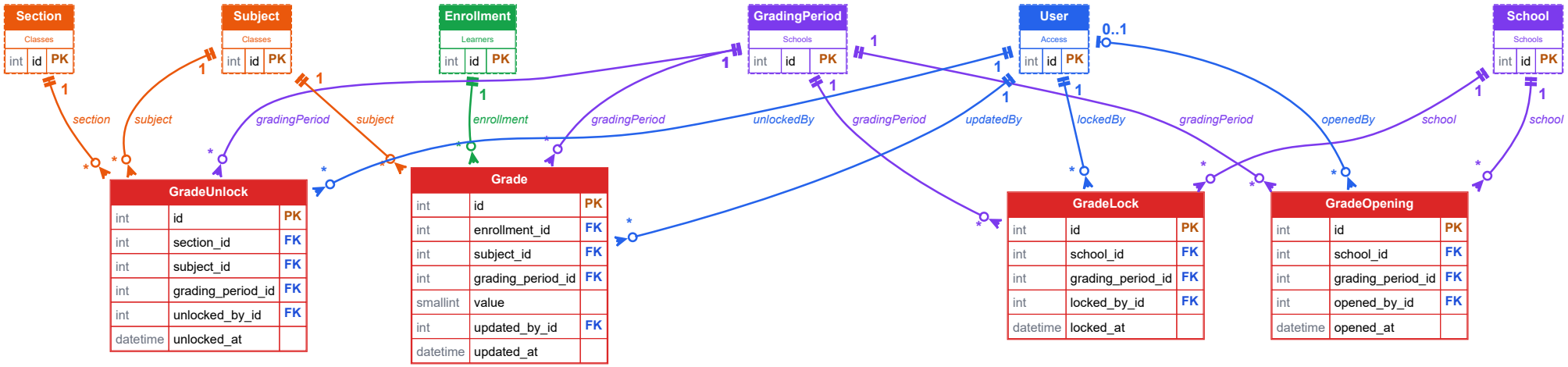
Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`section_id`	INT	FK	no	→ Section (on delete CASCADE)
`month`	DATE		no	
`submitted_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`submitted_at`	DATETIME		no	

Unique rules: `section_id, month`

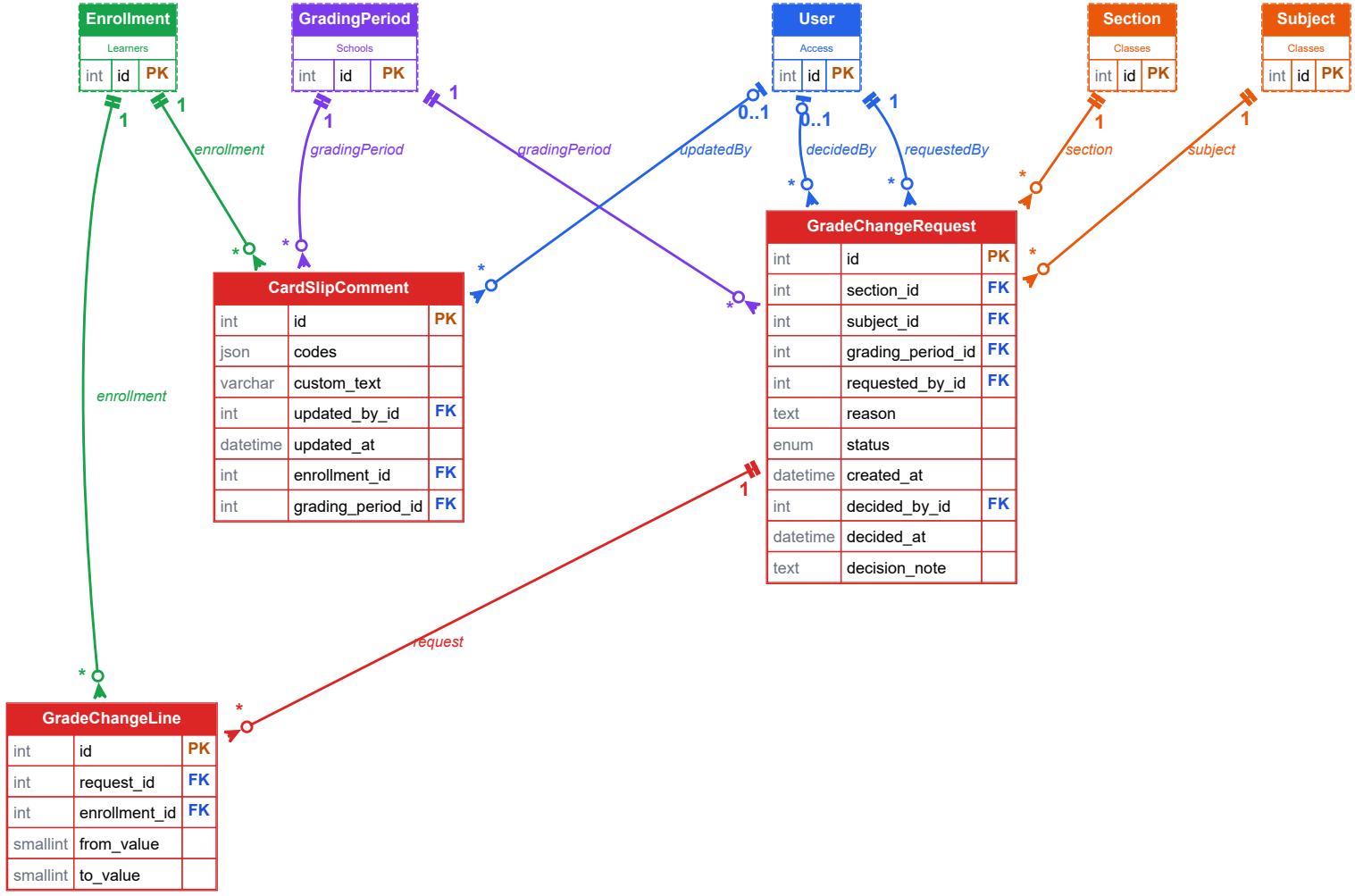
5.6 Grading: grades, openings, locks, change requests and card slips

Grades are per enrollment, subject and grading period. The school controls the period's state: a **GradeOpening** lets teachers enter grades; a **GradeLock** freezes them; a **GradeUnlock** re-opens one class for corrections. After a lock, teachers send a **GradeChangeRequest** (with **GradeChangeLines**) that the principal approves or declines. **CardSlipComment** holds the adviser's comments for report cards.

Grades and the period controls



Change requests and card slip comments



Access · Schools · Classes · Learners · Grading — Line colour = the module of the table the line points to (its 1 side).

CardSlipComment table `card_slip_comment` · Grading

The adviser's chosen comments (built-in codes plus one typed comment) for a learner's card slip in one term.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`codes`	JSON		no	
`custom_text`	VARCHAR(300)		yes	
`updated_by_id`	INT	FK	yes	→ User (on delete SET NULL)
`updated_at`	DATETIME		yes	
`enrollment_id`	INT	FK	no	→ Enrollment (on delete CASCADE)
`grading_period_id`	INT	FK	no	→ GradingPeriod (on delete CASCADE)

Unique rules: `enrollment_id, grading_period_id`

Grade table `grade` · Grading

A learner's whole-number grade (60-100) in one subject for one grading period. A blank grade has no row.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`enrollment_id`	INT	FK	no	→ Enrollment (on delete CASCADE)
`subject_id`	INT	FK	no	→ Subject (on delete RESTRICT)
`grading_period_id`	INT	FK	no	→ GradingPeriod (on delete CASCADE)
`value`	SMALLINT		no	
`updated_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`updated_at`	DATETIME		no	

Unique rules: `enrollment_id, subject_id, grading_period_id`

GradeChangeLine table `grade_change_line` · Grading

One learner's grade in a change request: what it was and what it should be.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`request_id`	INT	FK	no	→ GradeChangeRequest (on delete CASCADE)
`enrollment_id`	INT	FK	no	→ Enrollment (on delete CASCADE)
`from_value`	SMALLINT		yes	
`to_value`	SMALLINT		no	

Unique rules: `request_id, enrollment_id`

GradeChangeRequest table `grade_change_request` · Grading

A teacher's request to correct grades of a locked period, approved or declined by the principal (or withdrawn by the teacher).

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`section_id`	INT	FK	no	→ Section (on delete CASCADE)
`subject_id`	INT	FK	no	→ Subject (on delete RESTRICT)
`grading_period_id`	INT	FK	no	→ GradingPeriod (on delete CASCADE)
`requested_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`reason`	TEXT		no	
`status`	VARCHAR(10) enum GradeChangeStatus		no	
`created_at`	DATETIME		no	
`decided_by_id`	INT	FK	yes	→ User (on delete SET NULL)
`decided_at`	DATETIME		yes	
`decision_note`	TEXT		yes	

GradeLock table `grade_lock` · Grading

The principal's lock on a grading period: while it exists no grade of the school can change for that period.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`school_id`	INT	FK	no	→ School (on delete CASCADE)
`grading_period_id`	INT	FK	no	→ GradingPeriod (on delete CASCADE)
`locked_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`locked_at`	DATETIME		no	

Unique rules: `school_id, grading_period_id`

GradeOpening table `grade_opening` · Grading

The school's switch: while it exists, teachers can record grades for that period.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`school_id`	INT	FK	no	→ School (on delete CASCADE)
`grading_period_id`	INT	FK	no	→ GradingPeriod (on delete CASCADE)
`opened_by_id`	INT	FK	yes	→ User (on delete SET NULL)
`opened_at`	DATETIME		no	

Unique rules: `school_id, grading_period_id`

GradeUnlock table `grade_unlock` · Grading

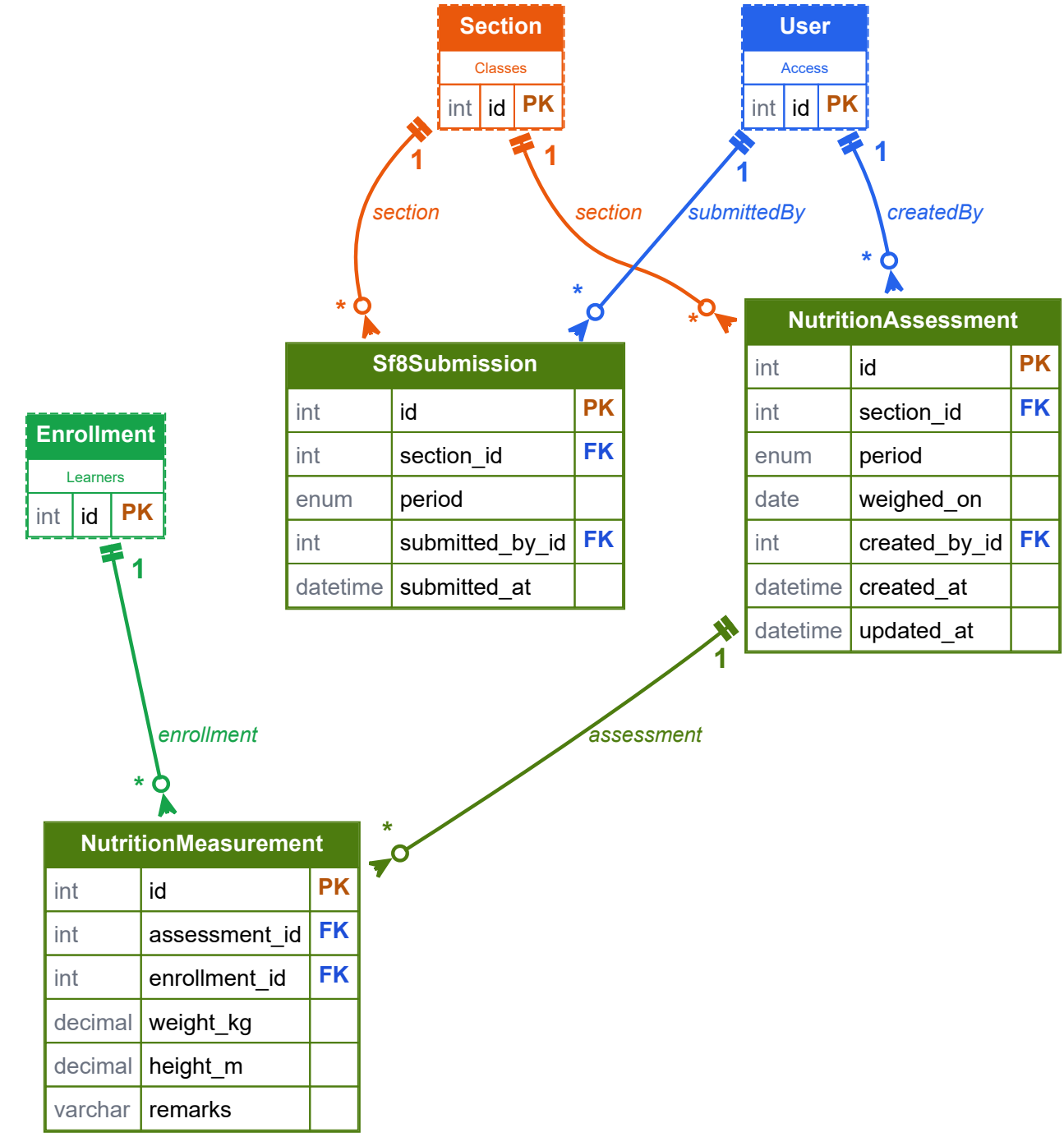
The principal's permission to correct one class's (section + subject) grades in a locked period.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`section_id`	INT	FK	no	→ Section (on delete CASCADE)
`subject_id`	INT	FK	no	→ Subject (on delete CASCADE)
`grading_period_id`	INT	FK	no	→ GradingPeriod (on delete CASCADE)
`unlocked_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`unlocked_at`	DATETIME		no	

Unique rules: `section_id, subject_id, grading_period_id`

5.7 Nutrition: weighings and SF8

Twice a year (baseline and endline) the adviser records a **NutritionAssessment** with a **NutritionMeasurement** per learner. BMI and nutritional status are computed from WHO tables at report time (not stored). Submitting creates an **Sf8Submission**, which locks the period.



Access · Classes · Learners · Nutrition — Line colour = the module of the table the line points to (its 1 side).

NutritionAssessment table `nutrition_assessment` · Nutrition

A section's baseline or endline weighing: the date it happened.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`section_id`	INT	FK	no	→ Section (on delete CASCADE)
`period`	VARCHAR(10) enum NutritionPeriod		no	
`weighed_on`	DATE		no	
`created_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`created_at`	DATETIME		no	
`updated_at`	DATETIME		yes	

Unique rules: `section_id, period`

NutritionMeasurement table `nutrition_measurement` · Nutrition

One learner's weight and height in a weighing (remarks may stand alone, e.g. absent).

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`assessment_id`	INT	FK	no	→ NutritionAssessment (on delete CASCADE)
`enrollment_id`	INT	FK	no	→ Enrollment (on delete CASCADE)
`weight_kg`	DECIMAL(5,2)		yes	
`height_m`	DECIMAL(3,2)		yes	
`remarks`	VARCHAR(100)		yes	

Unique rules: `assessment_id, enrollment_id`

Sf8Submission table `sf8_submission` · Nutrition

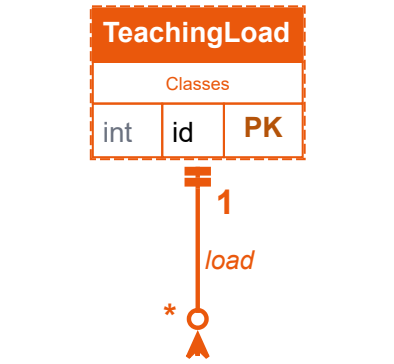
The adviser's SF8 (Nutritional Status) for one weighing period handed to the principal. While it exists, that period is locked.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`section_id`	INT	FK	no	→ Section (on delete CASCADE)
`period`	VARCHAR(10) enum NutritionPeriod		no	
`submitted_by_id`	INT	FK	no	→ User (on delete RESTRICT)
`submitted_at`	DATETIME		no	

Unique rules: `section_id, period`

5.8 Lesson plans

A **LessonPlan** is one week of a teaching load in the ILAW format. Its days (dates, holidays, per-day cells) are a JSON column, built from the timetable and the school's days off.



LessonPlan		
int	id	PK
int	load_id	FK
date	week_start	
varchar	topic	
text	competencies	
text	class_notes	
text	reference_list	
text	ai_declaration	
json	days	
datetime	created_at	
datetime	updated_at	

Classes LessonPlans Line colour = the module of the table the line points to (its 1 side).

LessonPlan table `lesson_plan` · LessonPlans

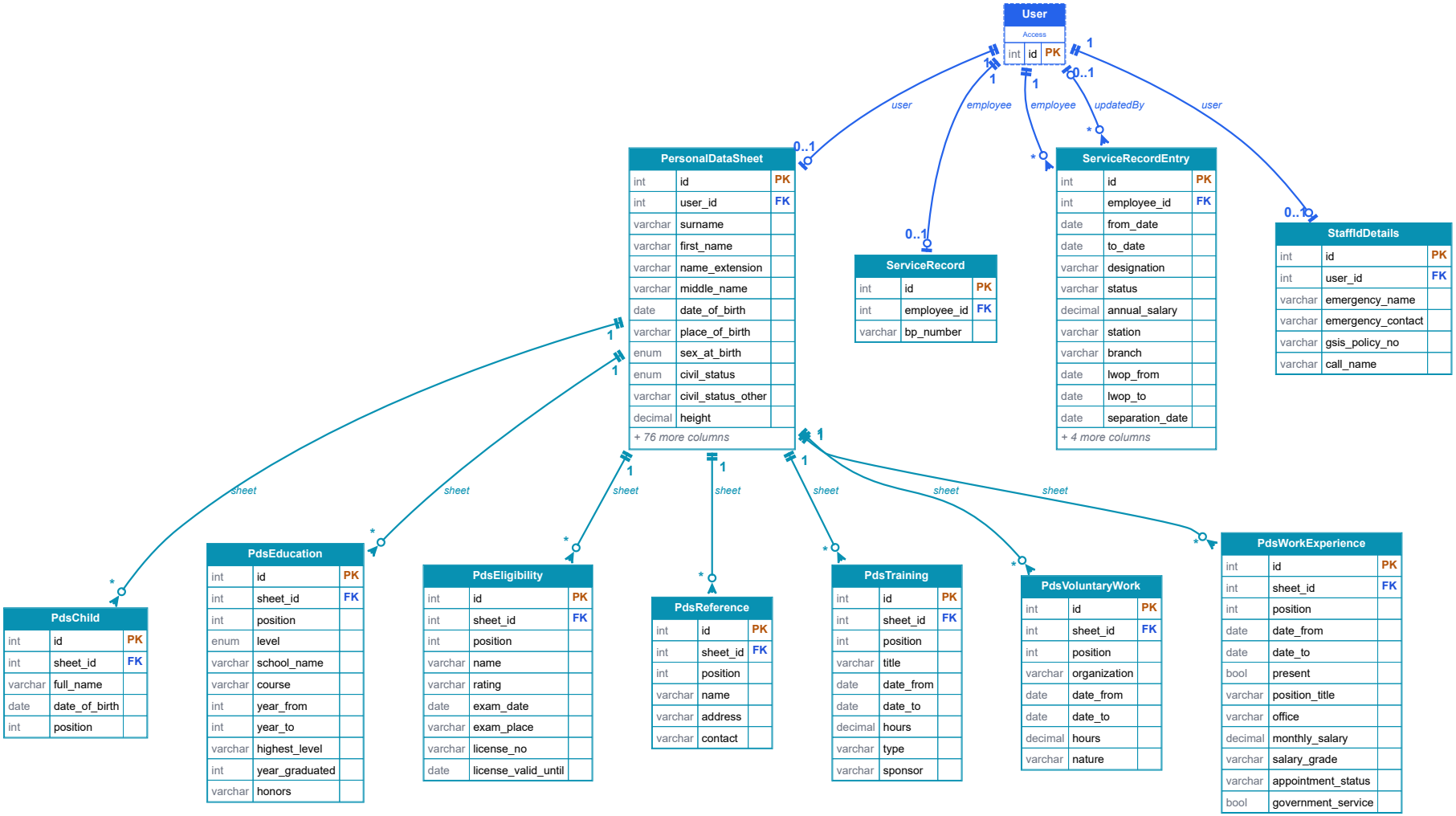
A teacher's weekly ILAW lesson plan for one teaching load. The day-by-day cells are stored as JSON.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`load_id`	INT	FK	no	→ TeachingLoad (on delete CASCADE)
`week_start`	DATE		no	
`topic`	VARCHAR(200)		no	
`competencies`	TEXT		no	
`class_notes`	TEXT		yes	
`reference_list`	TEXT		yes	
`ai_declaration`	TEXT		yes	
`days`	JSON		no	
`created_at`	DATETIME		no	
`updated_at`	DATETIME		no	

Unique rules: `load_id, week_start`

5.9 Staff: Personal Data Sheet, service record and staff ID

Personnel records for principals and teachers. The **PersonalDataSheet** (one per account) has seven child lists. The **ServiceRecord** and its **ServiceRecordEntries** are kept by the principal. **StaffIdDetails** completes what the staff ID card needs.



Access Staff Line colour = the module of the table the line points to (its 1 side).

PdsChild table `pds_child` · Staff

PDS item 23: one child.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`sheet_id`	INT	FK	no	→ PersonalDataSheet (on delete CASCADE)
`full_name`	VARCHAR(150)		no	
`date_of_birth`	DATE		yes	
`position`	INT		no	

PdsEducation table `pds_education` · Staff

PDS item 26: one school attended.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`sheet_id`	INT	FK	no	→ PersonalDataSheet (on delete CASCADE)
`position`	INT		no	
`level`	VARCHAR(20) enum EducationLevel		no	
`school_name`	VARCHAR(200)		no	
`course`	VARCHAR(200)		yes	
`year_from`	INT		yes	
`year_to`	INT		yes	
`highest_level`	VARCHAR(100)		yes	
`year_graduated`	INT		yes	
`honors`	VARCHAR(200)		yes	

PdsEligibility table `pds_eligibility` · Staff

PDS item 27: one civil service eligibility, board/bar or licence.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`sheet_id`	INT	FK	no	→ PersonalDataSheet (on delete CASCADE)
`position`	INT		no	
`name`	VARCHAR(200)		no	
`rating`	VARCHAR(20)		yes	
`exam_date`	DATE		yes	
`exam_place`	VARCHAR(200)		yes	
`license_no`	VARCHAR(50)		yes	
`license_valid_until`	DATE		yes	

PdsReference table `pds_reference` · Staff

PDS item 41: one reference (three on the form).

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`sheet_id`	INT	FK	no	→ PersonalDataSheet (on delete CASCADE)
`position`	INT		no	
`name`	VARCHAR(150)		no	
`address`	VARCHAR(255)		no	
`contact`	VARCHAR(100)		no	

PdsTraining table `pds_training` · Staff

PDS item 29: one training program.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`sheet_id`	INT	FK	no	→ PersonalDataSheet (on delete CASCADE)
`position`	INT		no	
`title`	VARCHAR(255)		no	
`date_from`	DATE		no	
`date_to`	DATE		yes	
`hours`	DECIMAL(6,2)		yes	
`type`	VARCHAR(50)		yes	
`sponsor`	VARCHAR(255)		yes	

PdsVoluntaryWork table `pds_voluntary_work` · Staff

PDS item 30: one voluntary work.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`sheet_id`	INT	FK	no	→ PersonalDataSheet (on delete CASCADE)
`position`	INT		no	
`organization`	VARCHAR(255)		no	
`date_from`	DATE		yes	
`date_to`	DATE		yes	
`hours`	DECIMAL(6,2)		yes	
`nature`	VARCHAR(150)		yes	

PdsWorkExperience table `pds_work_experience` · Staff

PDS item 28: one job, government or private.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`sheet_id`	INT	FK	no	→ PersonalDataSheet (on delete CASCADE)
`position`	INT		no	
`date_from`	DATE		no	
`date_to`	DATE		yes	
`present`	BOOLEAN		no	
`position_title`	VARCHAR(200)		no	
`office`	VARCHAR(255)		no	
`monthly_salary`	DECIMAL(12,2)		yes	
`salary_grade`	VARCHAR(10)		yes	
`appointment_status`	VARCHAR(50)		yes	
`government_service`	BOOLEAN		no	

PersonalDataSheet table `personal_data_sheet` · Staff

The Personal Data Sheet (CS Form 212) of a principal or teacher, one per account. Single-value items are columns; repeating items are child tables; skills/distinctions/memberships are JSON lists.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`user_id`	INT	FK	no	→ User (on delete CASCADE)
`surname`	VARCHAR(80)		yes	
`first_name`	VARCHAR(80)		yes	
`name_extension`	VARCHAR(10)		yes	
`middle_name`	VARCHAR(80)		yes	
`date_of_birth`	DATE		yes	
`place_of_birth`	VARCHAR(150)		yes	
`sex_at_birth`	VARCHAR(10) enum SexAtBirth		yes	
`civil_status`	VARCHAR(10) enum CivilStatus		yes	
`civil_status_other`	VARCHAR(50)		yes	
`height`	DECIMAL(4,2)		yes	
`weight`	DECIMAL(5,2)		yes	
`blood_type`	VARCHAR(5)		yes	
`umid_no`	VARCHAR(30)		yes	
`pagibig_no`	VARCHAR(30)		yes	

philhealth_no	VARCHAR(30)		yes	
philsys_pcn	VARCHAR(30)		yes	
tin	VARCHAR(30)		yes	
agency_employee_no	VARCHAR(30)		yes	
citizenship	VARCHAR(10) enum Citizenship		yes	
dual_citizenship_by	VARCHAR(20) enum DualCitizenshipBy		yes	
dual_citizenship_country	VARCHAR(80)		yes	
res_house_no	VARCHAR(50)		yes	
res_street	VARCHAR(100)		yes	
res_subdivision	VARCHAR(100)		yes	
res_barangay	VARCHAR(100)		yes	
res_city	VARCHAR(100)		yes	
res_province	VARCHAR(100)		yes	
res_zip	VARCHAR(4)		yes	
perm_same_as_residential	BOOLEAN		no	
perm_house_no	VARCHAR(50)		yes	
perm_street	VARCHAR(100)		yes	
perm_subdivision	VARCHAR(100)		yes	
perm_barangay	VARCHAR(100)		yes	
perm_city	VARCHAR(100)		yes	
perm_province	VARCHAR(100)		yes	
perm_zip	VARCHAR(4)		yes	
telephone_no	VARCHAR(30)		yes	
mobile_no	VARCHAR(20)		yes	
email	VARCHAR(180)		yes	
spouse_surname	VARCHAR(80)		yes	
spouse_first_name	VARCHAR(80)		yes	
spouse_name_extension	VARCHAR(10)		yes	
spouse_middle_name	VARCHAR(80)		yes	
spouse_occupation	VARCHAR(100)		yes	
spouse_employer	VARCHAR(150)		yes	
spouse_business_address	VARCHAR(255)		yes	
spouse_telephone	VARCHAR(30)		yes	
father_surname	VARCHAR(80)		yes	
father_first_name	VARCHAR(80)		yes	

`father_name_extension`	VARCHAR(10)		yes	
`father_middle_name`	VARCHAR(80)		yes	
`mother_maiden_surname`	VARCHAR(80)		yes	
`mother_first_name`	VARCHAR(80)		yes	
`mother_middle_name`	VARCHAR(80)		yes	
`skills`	JSON		no	
`distinctions`	JSON		no	
`memberships`	JSON		no	
`q34a`	BOOLEAN		yes	
`q34b`	BOOLEAN		yes	
`q34_details`	VARCHAR(255)		yes	
`q35a`	BOOLEAN		yes	
`q35a_details`	VARCHAR(255)		yes	
`q35b`	BOOLEAN		yes	
`q35b_details`	VARCHAR(255)		yes	
`q35b_date_filed`	DATE		yes	
`q35b_status`	VARCHAR(100)		yes	
`q36`	BOOLEAN		yes	
`q36_details`	VARCHAR(255)		yes	
`q37`	BOOLEAN		yes	
`q37_details`	VARCHAR(255)		yes	
`q38a`	BOOLEAN		yes	
`q38a_details`	VARCHAR(255)		yes	
`q38b`	BOOLEAN		yes	
`q38b_details`	VARCHAR(255)		yes	
`q39`	BOOLEAN		yes	
`q39_details`	VARCHAR(100)		yes	
`q40a`	BOOLEAN		yes	
`q40a_details`	VARCHAR(150)		yes	
`q40b`	BOOLEAN		yes	
`q40b_details`	VARCHAR(50)		yes	
`q40c`	BOOLEAN		yes	
`q40c_details`	VARCHAR(50)		yes	
`gov_id_type`	VARCHAR(80)		yes	
`gov_id_number`	VARCHAR(80)		yes	
`gov_id_issued`	VARCHAR(150)		yes	
`updated_at`	DATETIME		yes	

ServiceRecord table `service_record` · Staff

The parts of an employee's service record that are not rows: the GSIS BP number.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`employee_id`	INT	FK	no	→ User (on delete CASCADE)
`bp_number`	VARCHAR(30)		yes	

ServiceRecordEntry table `service_record_entry` · Staff

One row of an employee's service record (designation, status, salary, station, leave without pay, separation). Belongs to the person, not the school, so it follows them.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`employee_id`	INT	FK	no	→ User (on delete CASCADE)
`from_date`	DATE		no	
`to_date`	DATE		yes	
`designation`	VARCHAR(100)		no	
`status`	VARCHAR(50)		no	
`annual_salary`	DECIMAL(12,2)		no	
`station`	VARCHAR(150)		no	
`branch`	VARCHAR(20)		no	
`lwop_from`	DATE		yes	
`lwop_to`	DATE		yes	
`separation_date`	DATE		yes	
`separation_cause`	VARCHAR(150)		yes	
`remarks`	VARCHAR(150)		yes	
`updated_by_id`	INT	FK	yes	→ User (on delete SET NULL)
`updated_at`	DATETIME		no	

StaffIdDetails table `staff_id_details` · Staff

What a staff ID needs that the PDS lacks: emergency contact, GSIS policy number, call name.

Column	Type	Key	Null	Notes
`id`	INT	PK	no	
`user_id`	INT	FK	no	→ User (on delete CASCADE)
`emergency_name`	VARCHAR(100)		yes	
`emergency_contact`	VARCHAR(60)		yes	
`gsis_policy_no`	VARCHAR(30)		yes	
`call_name`	VARCHAR(30)		yes	

6. Enumerations

Fixed lists stored in columns. The PHP enum is in the module's `Domain/Model`.

Enum	Module	Values	Used by
LearnerMark	Attendance	absent, late, cutting	AttendanceMark
ShsTrack	Classes	academic, tvl_techpro, sports, arts_design	Section, Subject
Weekday	Classes	1 Monday ... 5 Friday	ScheduleEntry
ClassGradeStatus	Grading	open, closed, locked, unlocked	(rules only, not stored)
GradeChangeStatus	Grading	pending, approved, declined, withdrawn	GradeChangeRequest
LearningModality	Learners	face_to_face, modular_print, modular_digital, online, tv, radio, homeschooling, blended	Enrollment
SexAtBirth	Learners	male, female	Learner, PersonalDataSheet
Sf1Remark	Learners	transferred_in, dropped, late_enrollment, cct, balik_aral, sned, accelerated	Enrollment
TransferStatus	Learners	pending, approved, declined, cancelled	TransferRequest
NutritionPeriod	Nutrition	baseline, endline	NutritionAssessment, Sf8Submission
GradeLevel	Schools	0 Kinder, 1-12 Grade 1 to Grade 12	Section, Subject
GradingSystem	Schools	terms, quarters	SchoolYear
SchoolType	Schools	elementary, junior_high, senior_high, integrated	School
Citizenship	Staff	filipino, dual	PersonalDataSheet
CivilStatus	Staff	single, married, widowed, separated, others	PersonalDataSheet
DualCitizenshipBy	Staff	birth, naturalization	PersonalDataSheet
EducationLevel	Staff	elementary, secondary, vocational, college, graduate	PdsEducation

7. What happens on delete

Foreign keys say what the database does when the parent row is deleted:

Rule	Meaning	Where
CASCADE	Child rows are deleted with the parent (owned data).	AttendanceMark.rollCall, AttendanceMark.enrollment, RollCall.section, Sf2Submission.section, ClassPeriod.school, ClassPeriod.schoolYear, ScheduleEntry.schoolYear, Section.school, Section.schoolYear, Subject.school, Subject.schoolYear, Subject.parent, CardSlipComment.enrollment, CardSlipComment.gradingPeriod, Grade.enrollment, Grade.gradingPeriod, GradeChangeLine.request, GradeChangeLine.enrollment, GradeChangeRequest.section, GradeChangeRequest.gradingPeriod, GradeLock.school, GradeLock.gradingPeriod, GradeOpening.school, GradeOpening.gradingPeriod, GradeUnlock.section, GradeUnlock.subject, GradeUnlock.gradingPeriod, Enrollment.schoolYear, TransferRequest.fromSection, TransferRequest.toSection, LessonPlan.load, NutritionAssessment.section, NutritionMeasurement.assessment, NutritionMeasurement.enrollment, Sf8Submission.section, GradingPeriod.schoolYear, SchoolAssignment.schoolYear, SchoolDayOff.school, SchoolDayOff.schoolYear, PdsChild.sheet, PdsEducation.sheet, PdsEligibility.sheet, PdsReference.sheet, PdsTraining.sheet, PdsVoluntaryWork.sheet, PdsWorkExperience.sheet, PersonalDataSheet.user, ServiceRecord.employee, ServiceRecordEntry.employee, StaffIdDetails.user

RESTRICT	The parent cannot be deleted while children exist (history must stay).	RollCall.takenBy, Sf2Submission.submittedBy, ScheduleEntry.load, ScheduleEntry.section, ScheduleEntry.teacher, ScheduleEntry.period, TeachingLoad.section, TeachingLoad.subject, TeachingLoad.teacher, Grade.subject, Grade.updatedBy, GradeChangeRequest.subject, GradeChangeRequest.requestedBy, GradeLock.lockedBy, GradeUnlock.unlockedBy, Enrollment.learner, Enrollment.school, Enrollment.section, Enrollment.enrolledBy, Learner.user, TransferRequest.learner, TransferRequest.fromSchool, TransferRequest.requestedBy, NutritionAssessment.createdBy, Sf8Submission.submittedBy, SchoolDayOff.createdBy
SET NULL	The link is cleared; the child row stays (e.g. who decided, the author).	Announcement.author, Section.adviser, CardSlipComment.updatedBy, GradeChangeRequest.decidedBy, GradeOpening.openedBy, TransferRequest.enrollment, TransferRequest.decidedBy, School.idSignatureBy, ServiceRecordEntry.updatedBy
(none)	Database default: the parent cannot be deleted while children exist.	School.plan, SchoolAssignment.school, SchoolAssignment.user

> In practice records are rarely deleted: accounts and schools are *deactivated*, learners are *unenrolled* (the learner and past years stay), and teachers are taken off a school for the year.

8. Rules the database enforces (unique constraints)

Entity	Unique on	Business meaning
User	`username`	Unique value
AttendanceMark	`roll_call_id, enrollment_id`	One mark per learner per roll call
RollCall	`section_id, date`	One roll call per section per day
Sf2Submission	`section_id, month`	One SF2 per section per month
Plan	`name`	Unique value
ScheduleEntry	`section_id, day, period_id, term`	A section's slot holds one class
ScheduleEntry	`teacher_id, school_year_id, day, period_id, term`	A teacher can't be in two places at once
Section	`school_id, school_year_id, grade_level, name`	One name per grade level, school and year
Section	`school_year_id, adviser_id`	A teacher advises at most one section per year
Subject	`school_id, school_year_id, code`	One subject code per school and year
TeachingLoad	`section_id, subject_id`	One teacher per subject in a section
CardSlipComment	`enrollment_id, grading_period_id`	One comment set per learner per term
Grade	`enrollment_id, subject_id, grading_period_id`	One grade per learner, subject and period
GradeChangeLine	`request_id, enrollment_id`	One line per learner per request
GradeLock	`school_id, grading_period_id`	One lock per school per period

GradeOpening	`school_id, grading_period_id`	One opening per school per period
GradeUnlock	`section_id, subject_id, grading_period_id`	One unlock per class per period
Enrollment	`learner_id, school_year_id`	One enrollment per learner per school year
Learner	`lrn`	Unique value
LessonPlan	`load_id, week_start`	One plan per teaching load per week
NutritionAssessment	`section_id, period`	One weighing per section per period
NutritionMeasurement	`assessment_id, enrollment_id`	One measurement per learner per weighing
Sf8Submission	`section_id, period`	One SF8 per section per period
Division	`region, name`	Each region + division once
GradingPeriod	`school_year_id, number`	Period numbers are unique within a year
School	`school_id`	Unique value
SchoolAssignment	`user_id, school_year_id`	One assignment per user per school year
SchoolDayOff	`school_id, date`	A date is a day off once per school
SchoolYear	`name`	Unique value

9. Glossary

Term	Meaning
LRN	Learner Reference Number: the 12-digit DepEd ID of a learner, also the student's username.
School ID	The 6-digit DepEd ID of a school.
SF1	School Form 1, the School Register (class list with learner details and remarks).
SF2	School Form 2, the Daily Attendance Report of Learners, submitted monthly.
SF8	School Form 8, the Learner's Basic Health and Nutrition Report (baseline and endline weighings).
Card slip	The learner's report card for a term: grades, attendance and the adviser's comments.
ILAW	The weekly lesson plan format (Daily Lesson Log, ILAW format) used for lesson plans.
PDS	Personal Data Sheet, CS Form 212, the civil service personal record of an employee.
Service record	The employer's history of an employee's positions, status, salary and station.
SDS	Schools Division Superintendent; signs principals' IDs.
Adviser	The teacher in charge of a section (class list, attendance, SF2, SF8, card slips).
Teaching load	A teacher + subject + section assignment.
MAPEH components	Subjects with a parent (Music, Arts, PE, Health); the parent's grade is their average.
SHS track / strand	Senior High (Grades 11-12) specialisation: Academic, TVL, Sports, Arts & Design, plus a strand.
Seat / plan	A school buys seats on a plan; each seat allows a number of teachers and students.
Transaction lock	A named lock (e.g. `learner-12`) a use case takes so two saves on the same data can't overlap.